

Tematyka wykładów i zajęć laboratoryjnych

(oficjalny sylabus, kolejność i tematyka może być nieco inna)

1. Wstęp. Urządzenia graficzne wejścia-wyjścia.
2. Podstawowe pojęcia i definicje dotyczące obrazów cyfrowych (w technice rastrowej i wektorowej) oraz modeli kolorów.
3. Punktowe, lokalne i globalne metody przekształcania obrazów.
4. Transformacje geometryczne obiektów graficznych i wypełnianie figur.
5. Eliminacja elementów zasłoniętych, metody kreślenia krzywych.
6. Modele oświetlenia i cieniowania.
7. Przekształcenia morfologiczne i ich zastosowania.
8. Transformaty obrazów i sygnałów dźwiękowych (FFT, Hough, Radon).
Wybrane metody filtracji danych jedno- i dwuwymiarowych.
- 9. Wybrane modele interakcji człowiek-komputer. Zasady projektowania i analizy interfejsów komputerowych.**
10. Interfejsy dla osób niepełnosprawnych. Interakcja człowiek-komputer dla komputerów przyszłości.

GRAFIKA KOMPUTEROWA I KOMUNIKACJA Z KOMPUTEREM

Wykład 12

Andrzej Śluzek

pokój: 3/64

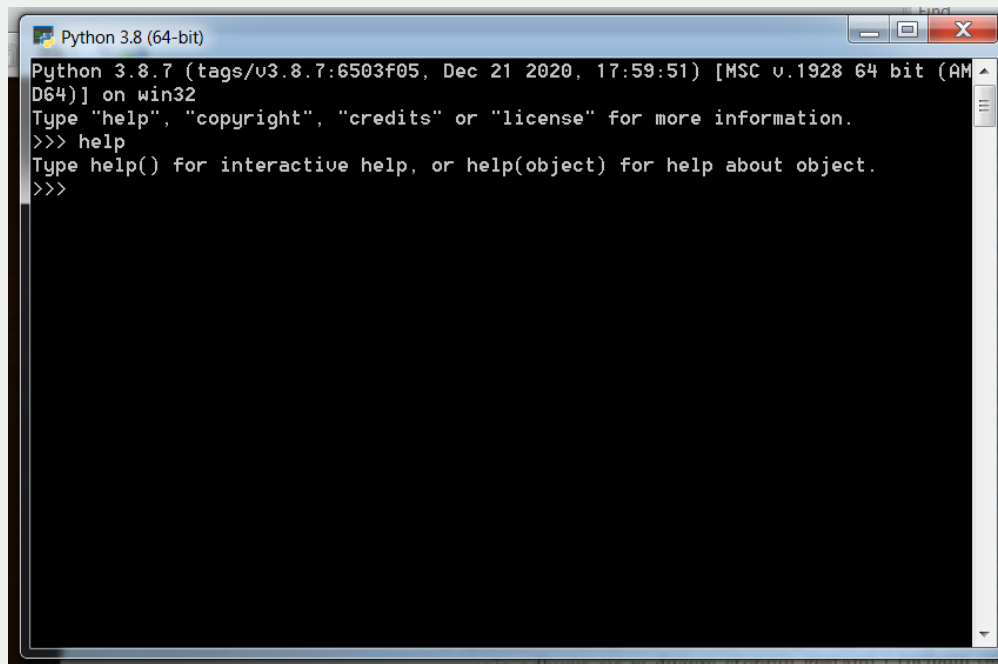
email: andrzej_sluzek@sggw.edu.pl



Modele interakcji

Historycznie, modele interakcji między człowiekiem i komputerem ewoluowały w następujący sposób:

- **Brak interakcji** (wsadowy model działania komputera).
- **Interakcja tekstowa** (np. *command window*). Ograniczone/utrudnione możliwości interaktywnego przekazywania danych (zwłaszcza wyników). Najczęściej realizowane w postaci poleceń jednowierszowych (*command line interface*).

A screenshot of a Windows command window titled "Python 3.8 (64-bit)". The window has a blue title bar and standard Windows window controls (minimize, maximize, close). The background is black with white text. The text displayed is the help output for the Python interpreter, showing the version (3.8.7), build information, and instructions on how to use the help command. The prompt is >>>.

```
Python 3.8 (64-bit)
Python 3.8.7 (tags/v3.8.7:6503f05, Dec 21 2020, 17:59:51) [MSC v.1928 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> help
Type help() for interactive help, or help(object) for help about object.
>>>
```

Modele interakcji

- **Interakcja oparta na ikonach** lub przyciskach (*icon-modal style* lub, w prostszej wersji, *key-modal style*). W tym modelu mieszczą się najróżniejsze warianty interakcji, ale wspólną ich cechą jest wybór spośród skończonej liczby aktualnie dostępnych opcji (ikonki, klawisze funkcyjne, skróty klawiszowe, itp.).



Modele interakcji

- **Interakcja wykorzystująca manipulację obiektami.** Obejmuje kilka różnych modeli (zwykle współistniejących), w których użytkownik manipuluje obiektami graficznymi w celu wykonania wybranych operacji lub pobrania ich wyników. W ramach tego rodzaju interakcji można wyróżnić następujące kategorie:

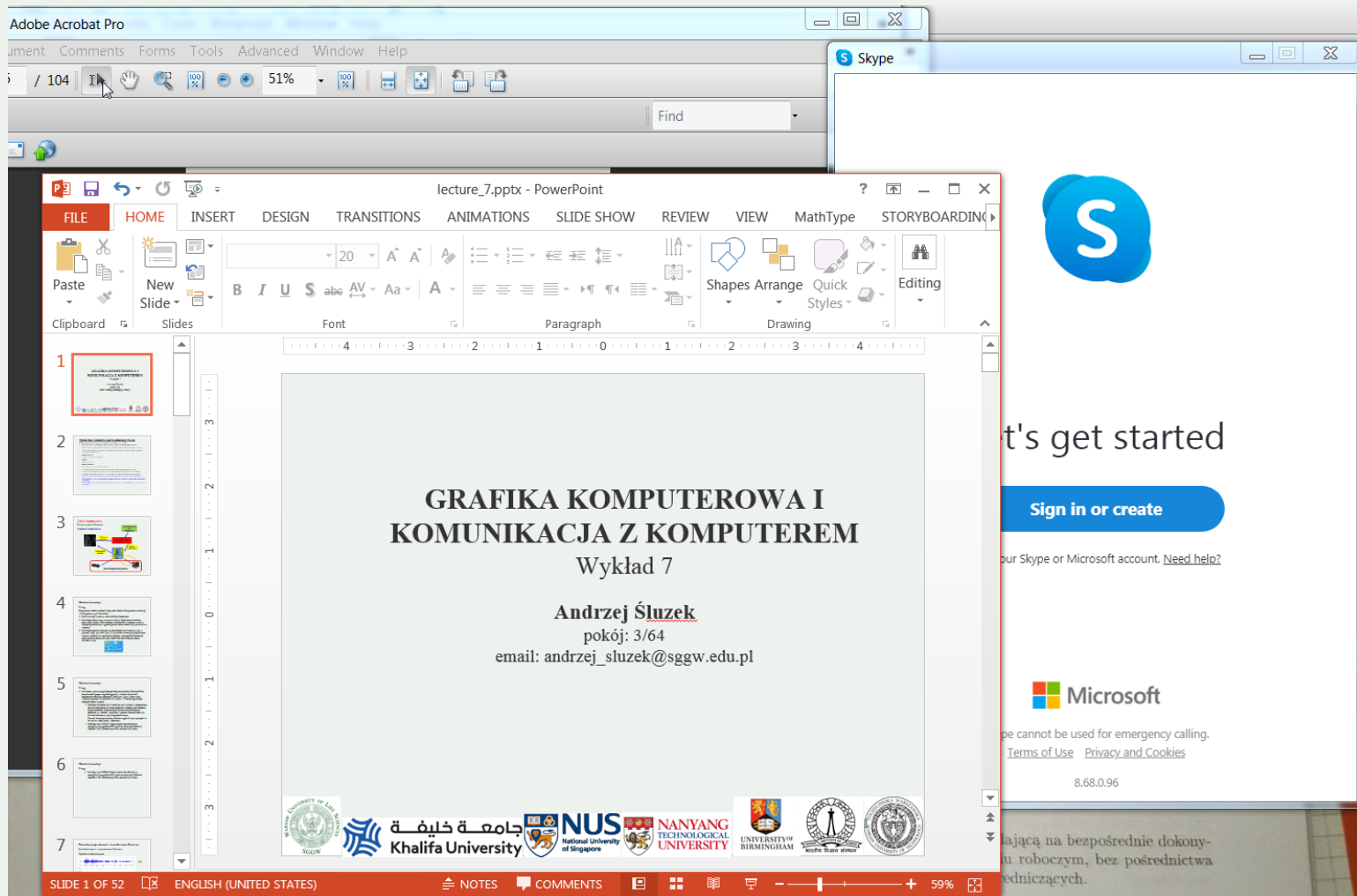
- ✓ **Interfejsy graficzne** (GUI – *graphical user interface*) wykorzystujące okna do przełączania się między zadaniami, przekazywania obiektów między zadaniami, skupiania się na aktualnie najważniejszych zadaniach, itp. Metoda „współpracy” z oknami może być różna (np. klawisze kierunkowe, mysz, bezpośredni dotyk).

Pierwsze komercyjne systemy okienkowe pojawiły się na początku lat 80-tych XX wieku (Xerox, Macintosh).

- ✓ **Interfejsy typu WIMP** (*Windows-Icons-Menu-Pointer*) są szczególnym przypadkiem GUI, gdzie do manipulacji obiektami (ikonami) i ich wybierania używa się pointerów (np. mysz).

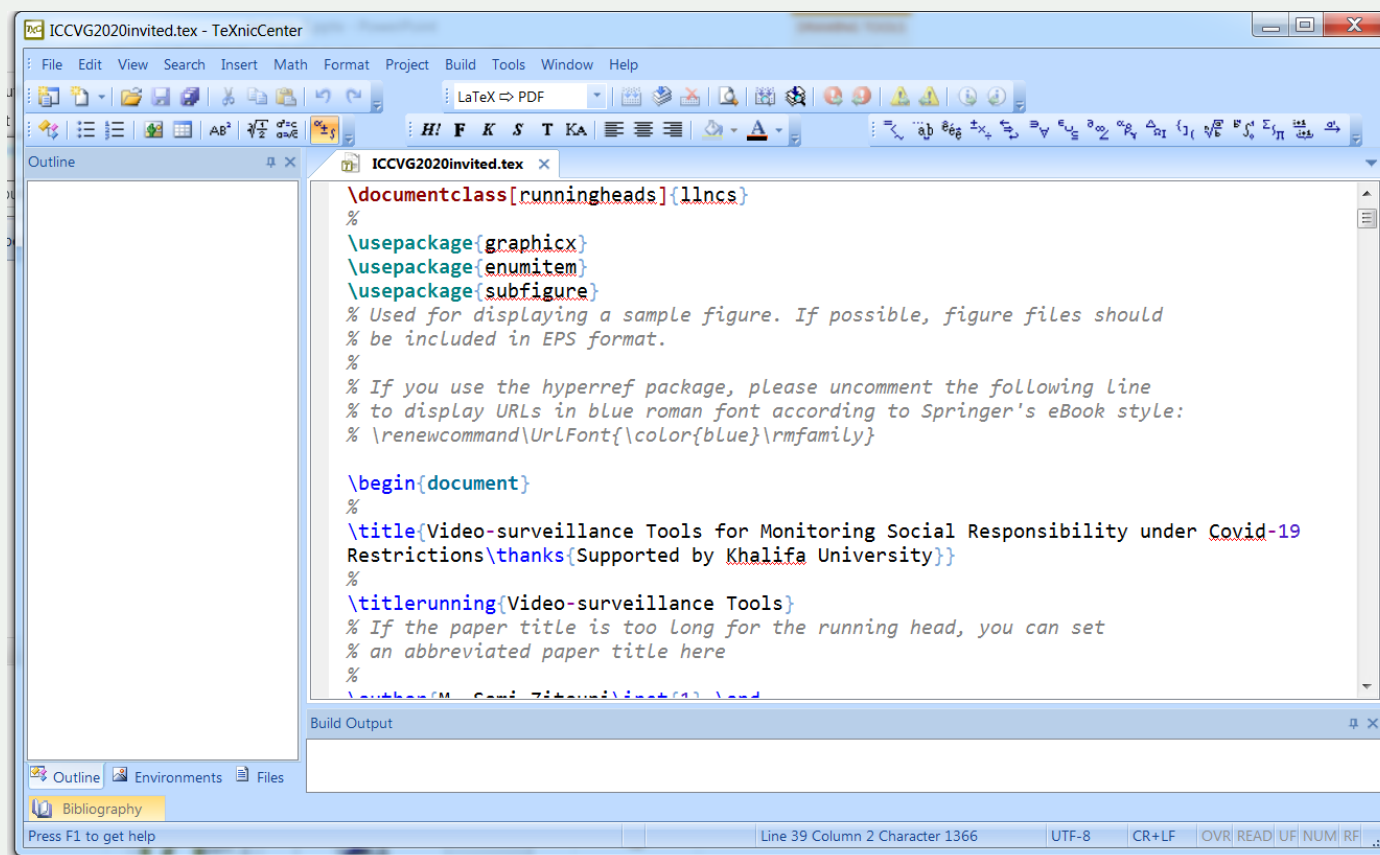
Modele interakcji

Przykład: GUI w połączeniu z WIMP



Modele interakcji

- ✓ Interfejsy typu **WYSIWYG** (*What-You See-Is-What-You-Get*) umożliwiają bezpośrednio obserwację wyników bez konieczności otwierania/uruchamiania innej aplikacji.



To nie jest
WYSIWYG

Modele interakcji

Edytor nie-WYSIWYG, np. **Latex**

```
\documentclass{llncs}
```

```
%
```

```
\usepackage{graphicx}
```

```
%
```

```
\begin{document}
```

```
\normalsize
```

```
Wz\or na znalezienie rozwiaza\n r\ownania kwadratowego: $ax^2+bx+c=0$:
```

```
\begin{equation}
```

```
x_{1,2}=\frac{-b\pm \sqrt{b^2-4ac}}{2}
```

```
\end{equation}
```

```
\begin{figure}[ht!b]
```

```
\begin{center}
```

```
\includegraphics[width=3cm]{pingu.jpg}
```

```
\end{center}
```

```
\end{figure}
```

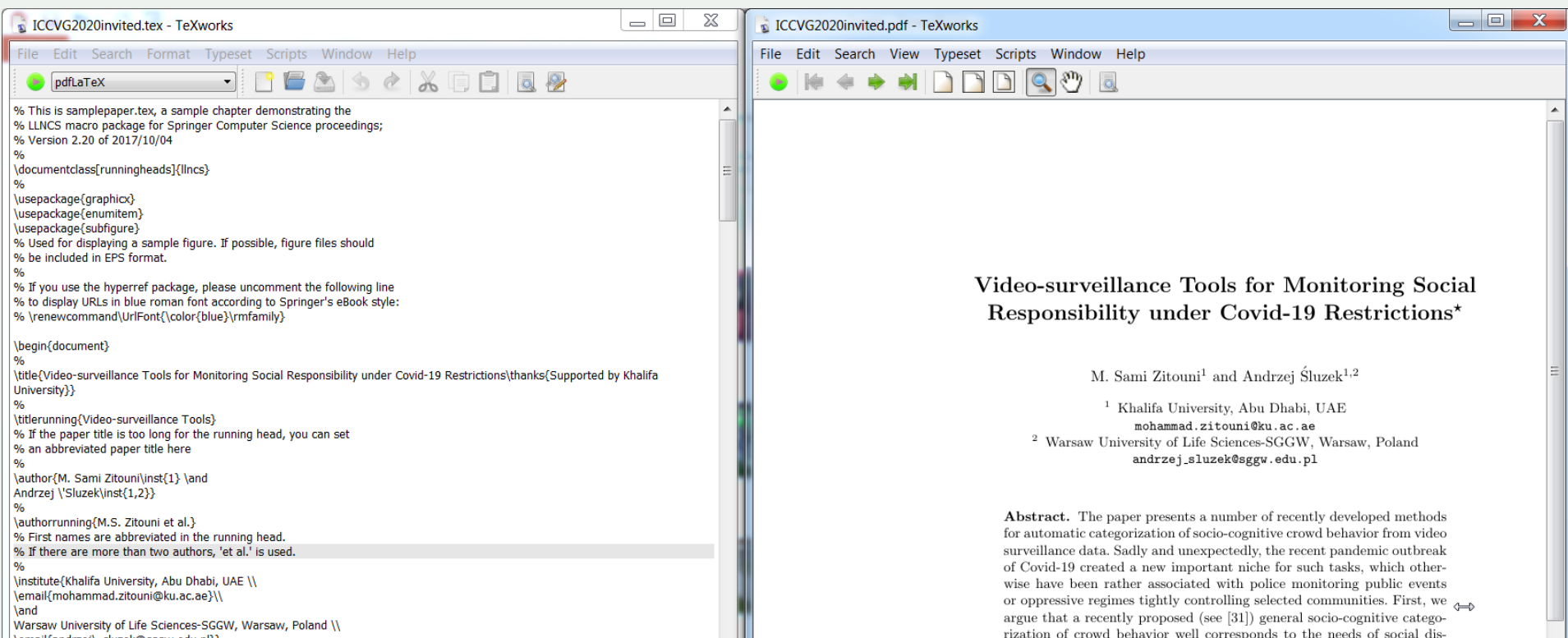
```
\end{document}
```

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2}$$



Modele interakcji

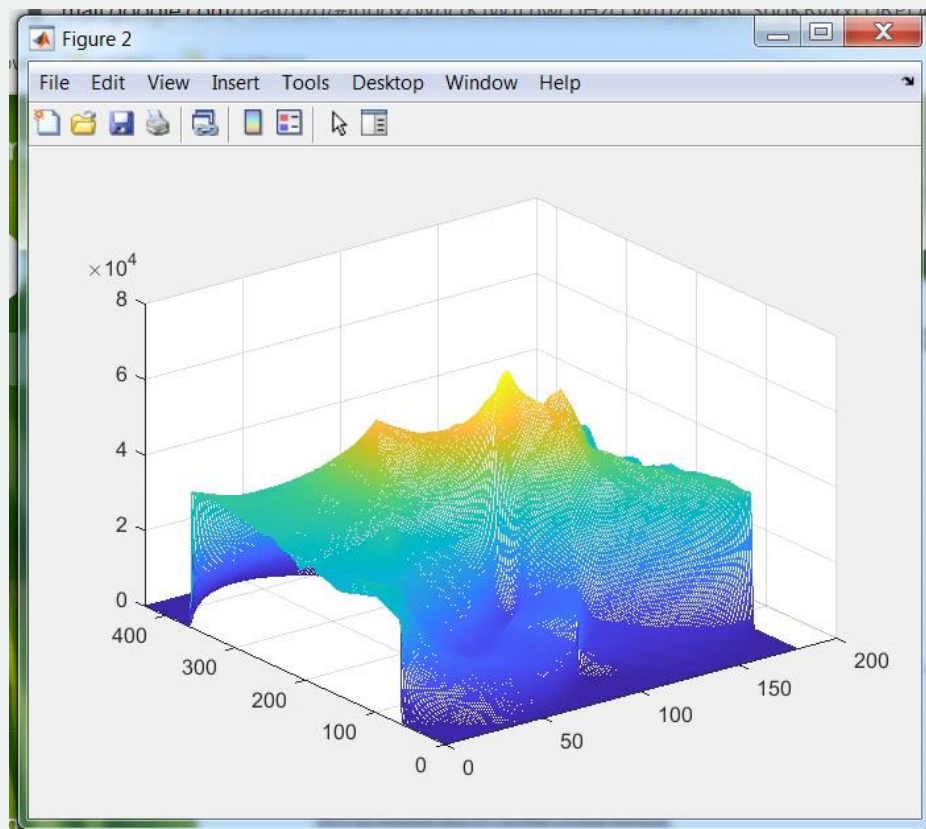
- ✓ Interfejsy typu WYSIWYG (*What-You See-Is-What-You-Get*) umożliwiają bezpośrednio obserwację wyników bez konieczności otwierania/uruchamiania innej aplikacji.



To jest WYSIWYG

Modele interakcji

- ✓ **Interfejsy z użyciem manipulacji bezpośredniej** umożliwiające bezpośrednio obserwacje wyników/danych bez konieczności otwierania/uruchamiania innej aplikacji.



Fizyczna metoda manipulacji (mysz, ekran dotykowy, joystick, itp.) ma drugorzędne znaczenie.

Modele interakcji

Interakcja wykorzystująca naturalne metody komunikacji. Obejmuje kilka różnych technik, które w maksymalny sposób starają się odwoływać do metod komunikacji występujących w świecie rzeczywistym. Najbardziej typowe warianty takich interakcji to:

- ✓ **Interakcja głosowa** występująca w wersji jednokierunkowej (synteza komunikatów głosowych przez komputer) lub w znacznie trudniejszej wersji dwukierunkowej (która dodatkowo wymaga systemu rozpoznawania mowy). Stosowana jest, na przykład, do interakcji z „inteligentnymi” robotami lub smartfonami.



Modele interakcji

- ✓ **Interakcja kinetyczno-haptyczna** obejmująca różnego rodzaju „zanurzenia” w rzeczywistości wirtualnej i komunikowanie się z nią przez gestykulację (wymagany system wizyjny do analizy ruchów ciała, np. Kinect) bądź dodatkowe dołączenie systemów haptycznych (dotykowych) dających jedno- lub dwukierunkowe „odczuwanie” dotyku i innych fizycznych interakcji (wymagane są złożone systemy elektromechaniczne).



Modele interakcji

- ✓ **Interakcja „rozmyta”** (wykorzystująca tzw. *ubiquitous computing*, czyli „wszechobecne moce obliczeniowe”). Różnorodne czujniki/systemy rozmieszczone na ludzkim ciele, lub w pobliżu człowieka, umożliwiają wymianę informacji z systemami komputerowymi na poziomie porównywalnym z komunikacją międzyludzką (a nawet bardziej zaawansowanym, np. „czytanie” intencji lub myśli użytkownika).
- ✓ **Interakcja multipersonalna.** Zanurzenie wybranych modeli interakcji naturalnej nie jest ograniczone do pojedynczych użytkowników. Wieloosobowa interakcja z tą samą przestrzenią wirtualną (albo z rzeczywistością rozszerzoną, ang. *augmented reality*) umożliwia tworzenie interfejsów o nie w pełni jeszcze przewidywalnych możliwościach (a również potencjalnych zagrożeniach). Ten rodzaj interakcji jest podstawą systemów typu *metaverse*.

Zasady projektowania interfejsów graficznych

Zakres tematów

- Trochę o metodyce **UCD** (*user-centered design*).

User-centered design (UCD) jest iteracyjnym podejściem do procesu projektowania. W metodzie tej projektant skupia się na potrzebach użytkownika w każdej fazie procesu.

W szczególności zespoły projektowe włączają użytkowników w proces projektowania używając różnych metod tak, aby powstał produkt w pełni użyteczny i łatwy w eksploatacji.

- Wskazówki dotyczące wizualnych aspektów **interfejsów graficznych** (GUI – *graphical user interface*).

Nawiązanie do *heurystyk Nielsena* oraz *heurystyk Shneidermana* (zapewnienie funkcjonalności i ergonomii).

Zasady projektowania interfejsów graficznych

Podstawy metodyki UCD (*user-centered design*)

UCD to iteracyjny proces projektowania (dowolnych produktów), gdzie w każdej fazie projektu na pierwszym planie są potrzeby użytkowników.

Zespół projektowy angażuje potencjalnych użytkowników w trakcie tworzenia produktu, wykorzystując w tym celu różne techniki i metody.

Każda iteracja procesu USD składa się z czterech (istotnie różnych od siebie) etapów:

- A. Zidentyfikowanie kontekstu w jakim użytkownicy będą wykorzystywać produkt.
- B. Zdefiniowanie wymagań.
- C. Zaprojektowanie rozwiązań.
- D. Porównanie (ewaluacja) jak produkt sprawdza się w kontekście zastosowań i czy spełnia wymagania tych zastosowań.

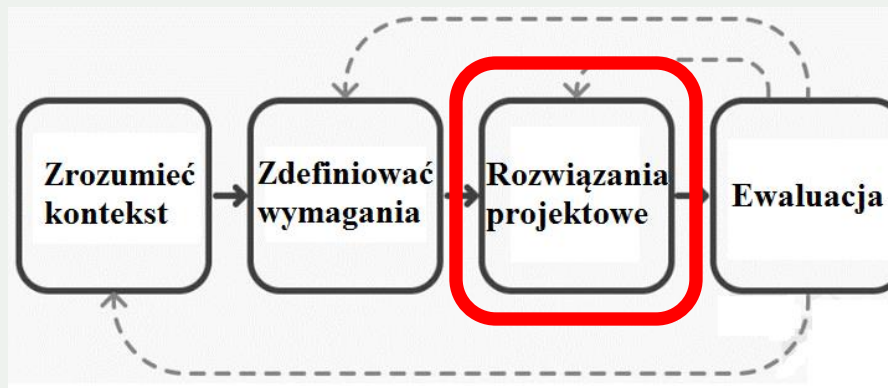
W punkcie (D) możemy wracać do (B) lub (C), jeśli zidentyfikowane zostaną niedociągnięcia.

A potem zaczynamy nową iterację, ponownie startując z punktu (A).

Zasady projektowania interfejsów graficznych

Podstawy metodyki UCD (*user-centered design*)

UCD to iteracyjny proces projektowania, gdzie w każdej fazie projektu na pierwszym planie są potrzeby użytkowników.



W UCD, projekty bazują na zrozumieniu *użytkowników, zadań i środowiska*, a głównym celem jest poznanie i odniesienie się do doświadczeń użytkowników we wszystkich trzech aspektach. Wymagać to może zaangażowania ekspertów z innych dziedzin niż zakres projektu.

Dwa najważniejsze punkty to:

- W każdym etapie powinni być zaangażowani potencjalni użytkownicy.
- Doświadczenia/oceny użytkowników powinny być gromadzone w wystarczająco długim i zróżnicowanym cyklu badań.

Zasady projektowania interfejsów graficznych

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- **Zastosowanie**
- **Funkcjonalność i ergonomia**
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- Charakterystyka nawyków i typowych źródeł błędów

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów graficznych

Wymagania wynikające z charakteru zadania

Zastosowanie projektowanego systemu (typowe problemy do rozważenia)

- Szybkość interakcji (czas rzeczywisty?, ograniczony czas reakcji?, prezentacja wyników obliczeń?)
- Bezpieczeństwo danych i procesów (nieistotne?, krytyczny wymóg?, na ile istotna jest bezbłędna reakcja użytkownika?)

Funkcjonalność i ergonomia (często oceniana wg tzw. heurystyk Nielsena, 1990-94)

- status systemu zawsze widoczny (możliwie najdogodniej dla użytkownika)
- swoboda działania i sprawianie wrażenie, że system jest pod kontrolą (np. przewidywalny czas reakcji, dostępność do całej „przestrzeni roboczej”)
- spójność (te same metody i standardy we wszystkich fragmentach/etapach)
- ułatwione rozpoznawanie (np. ikon) zamiast ciągłego przypominania ich znaczenia
- przewidywanie sytuacji prowadzących do powstawania błędów i zapobieganie takim sytuacjom
- właściwa terminologia (nie wykraczająca poza wiedzę użytkownika)
- elastyczne metody interakcji dopasowane do wymagań użytkownika (np. skróty)
- estetyka i „oszczędność” graficzna
- diagnostyka i korekcja błędów (czytelny opis błędu, operacja *undo*)
- dokumentacja i pomoc (łatwo dostępna pomoc na żądanie)

Zasady projektowania interfejsów graficznych

Wymagania wynikające z charakteru zadania

Funkcjonalność i ergonomia (alternatywne tzw. złote zasady Shneidermana, 1986)

- Dążenie do spójności.
- Umożliwienie doświadczonym użytkownikom korzystanie ze skrótów.
- Maksymalizacja informacji zwrotnej.
- Interakcja podkreślająca początek, trwanie i zakończenie realizowanych zadań.
- Prosta obsługa błędów (i minimalizacja ryzyka poważnych błędów).
- Łatwe przywracanie wyników sprzed wykonanych czynności.
- Dawanie poczucia, że to użytkownik kontroluje system.
- Minimalizacja obciążenie pamięci krótkotrwałej.

Zasady projektowania interfejsów graficznych

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- **Zgodność ze standardami (formalnymi i nieformalnymi)**

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- Charakterystyka nawyków i typowych źródeł błędów

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów graficznych

Wymagania wynikające z charakteru zadania

Zgodność ze standardami (formalnymi i nieformalnymi)

- zgodność z normami międzynarodowymi określającymi wymogi wobec produktów informatycznych, np. norma ISO/IEC 9126:
 - zrozumiałość (ang. *understandability*)
 - użyteczność (ang. *usability compliance*)
 - możliwość nauczania się (ang. *learnability*)
 - łatwość obsługi (ang. *operability*)
 - atrakcyjność (ang. *attractiveness*)
- lub norma ISO 9241:
 - skuteczność (ang. *effectiveness*)
 - efektywność (ang. *efficiency*)
 - satysfakcja (ang. *satisfaction*)
- zgodność ze standardami środowiska systemowego

Zasady projektowania interfejsów graficznych

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- **Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)**
- **Możliwości percepcyjne i pamięciowe użytkownika**
- Charakterystyka nawyków i typowych źródeł błędów

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów graficznych

Profil użytkownika

Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)

- profilowanie użytkowników wg tzw. *modelu Rasmussena*
 - działanie oparte na regułach (ang. *rule-based*) z wykorzystaniem prostych kryteriów typu *if.... then...* (gorzej wyszkoleni użytkownicy)
 - działanie oparte na odruchach (ang. *skill-based*) charakteryzujące dobrze wyszkolonych operatorów działających w znanej przestrzeni informatycznej
 - działanie oparte na wiedzy (ang. *knowledge-based*) zakładające znaczną kreatywność i elastyczność w podejmowaniu decyzji

Zasady projektowania interfejsów graficznych

Profil użytkownika

Możliwości percepcyjne i pamięciowe użytkownika

- modele pamięci i percepcji
 - pamięć krótko- i długoterminowa (zmiany w zależności od wieku i stanu umysłowości)
 - percepcja, np. rola sygnałów dźwiękowych (natężenie, barwa, częstotliwość), dobór formy komunikatów wizualnych (więcej później)
 - „nasycenie” zmysłów i uwagi (problem zmęczenia, dekoncentracji lub znudzenia)
 - ograniczona precyzja ruchów (np. błędy przy manipulacji kursorem), tzw. prawo Fittsa

Zasady projektowania interfejsów graficznych

Profil użytkownika

Możliwości percepcyjne i pamięciowe użytkownika

„Nasycenie” zmysłów (zasada magicznej siódemki).

Zasada ta opisuje ustalony empirycznie (i niekoniecznie prawdziwy w przypadku każdej osoby) fakt, że przy pojawieniu się więcej niż SIEDMIU niezależnych elementów informacyjnych (np. obiektów graficznych) następuje „nasycenie” zmysłów i analiza takiej informacji trwa znacznie dłużej.

Przykład „nasycenia”:

NBPPKOPZUNFZ

Ten sam przykład zrealizowany prawidłowo:

NBP PKO PZU NFZ

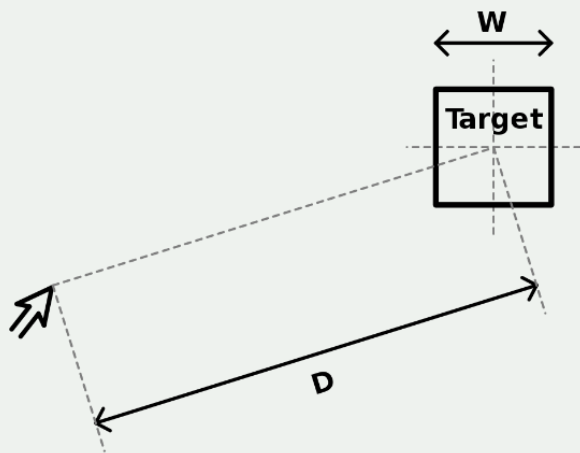
Zasady projektowania interfejsów graficznych

Profil użytkownika

Możliwości percepcyjne i pamięciowe użytkownika

Prawo Fittsa to próba skwantyfikowania ograniczonych możliwości percepcyjno-ruchowych przeciętnego użytkownika manipulującego kursorem (np. na ekranie).

Prawo to postuluje, że szybkie i zarazem precyzyjne ruchy nie są jednocześnie możliwe. Poniższa zależność aproksymuje czas potrzebny do „trafienia” w obiekt o ograniczonych rozmiarach:



$$t = x + y \log_2 \left(\frac{D}{W} + 1 \right)$$

gdzie x i y to stałe wyznaczone eksperymentalnie

Zasady projektowania interfejsów graficznych

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- **Charakterystyka nawyków i typowych źródeł błędów**

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów graficznych

Profil użytkownika

Charakterystyka nawyków i typowych źródeł błędów

- zachowania rutynowe (działanie „na pamięć” w niewłaściwych sytuacjach)
- błędy w wyniku wysokiego poziomu stresu w sytuacjach krytycznych
- podzielność uwagi (konieczna lub nawykowa)
- potencjalna sprzeczność kryteriów decyzyjnych
- przybliżony model nabieranie wprawy, tzw. *potęgowe prawo wprawy* (t_1 , to czas pierwszej próby)

$$t_n = t_0 + t_1 n^{-\lambda}$$

Zasady projektowania interfejsów graficznych

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- Charakterystyka nawyków i typowych źródeł błędów

3. Realizacja projektu

- **Sposoby pozyskiwania i przekazywania informacji**
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów graficznych

Realizacja projektu

Sposoby pozyskiwania i przekazywania informacji (realizacja dialogu)

Typowe obiekty:

- Polecenia wierszowe (interakcja tekstowa)
- Pytanie-odpowieź
(do you like cheese? Y/N ... You said YES. Are you sure? Y/N ...)
- Przyciski funkcyjne



- Formularze (głównie zastosowania biznesowe)

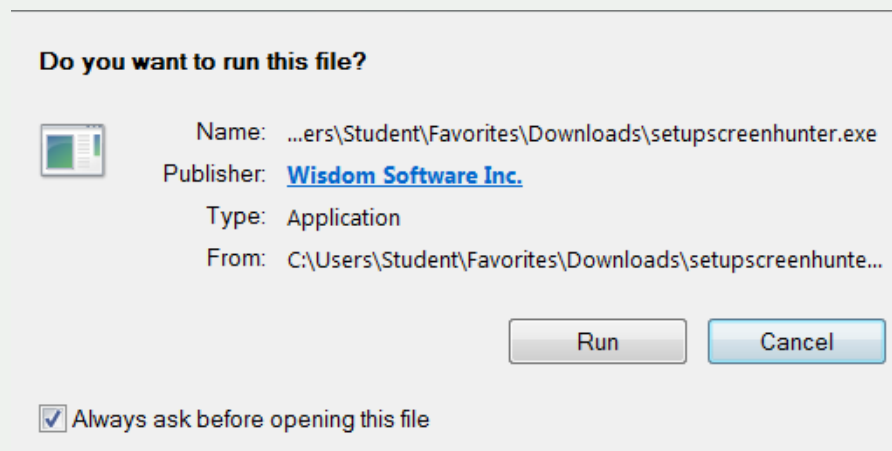
Zasady projektowania interfejsów graficznych

Realizacja projektu

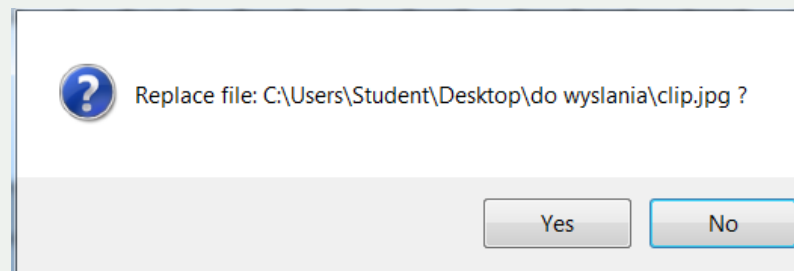
Sposoby pozyskiwania i przekazywania informacji (realizacja dialogu)

Typowe obiekty:

- Okna dialogowe



- Komunikaty informacyjne



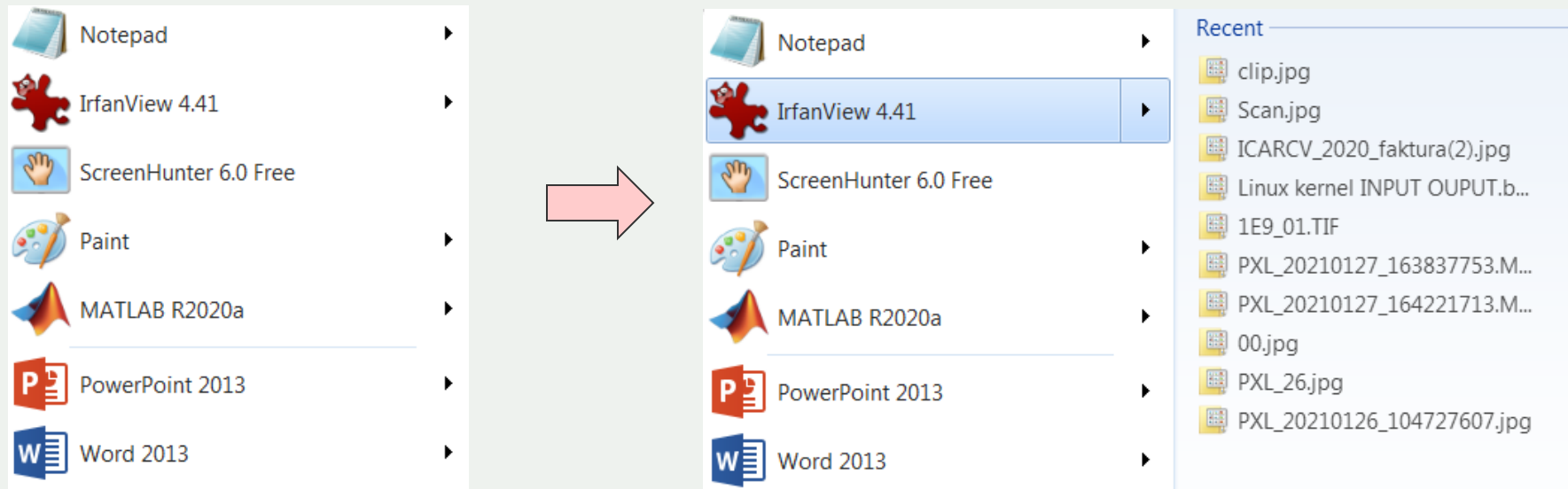
Zasady projektowania interfejsów graficznych

Realizacja projektu

Sposoby pozyskiwania i przekazywania informacji (realizacja dialogu)

Typowe obiekty:

- Menu (w tym menu hierarchiczne, kaskadowe, wstążkowe, palety, zakładki, itp.)



Zasady projektowania interfejsów graficznych

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- Charakterystyka nawyków i typowych źródeł błędów

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- **Wykorzystanie istniejących rozwiązań**
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika

Zasady projektowania interfejsów graficznych

Realizacja projektu

Wykorzystanie istniejących rozwiązań i wzorców (np. metafory)

Rola metafory (symbolicznej reprezentacji informacji) jest bardzo istotna, gdyż pozwala ona przekazywać niezbędną informację w możliwie najbardziej zwarty sposób powinna.

Metafora (np. forma graficzna ikony) powinna spełniać następujące wymagania :

- Jest czytelna znaczeniowo i graficznie (elementy wiedzy o kulturze potencjalnych użytkowników).
- Jest oszczędna (odwoływać się tylko do najważniejszych własności reprezentowanych obiektów).
- Pozbawiona jest nieistotnych „ozdobników”.
- Reprezentuje tylko obiekty/własności niezbędne dla wykonywanego zadania.
- Jest spójna znaczeniowo (w systematyczny sposób reprezentuje podobne obiekty w całym systemie). Być może należy korzystać z istniejących metafor.
- Nie sprawia wrażenia przestarzałej (np. taśma magnetofonowa jako metafora nagrywania dźwięku).
- Przyciąga uwagę użytkownika (więcej na kolejnych slajdach).

Zasady projektowania interfejsów graficznych

Podstawy percepcji wizualnej

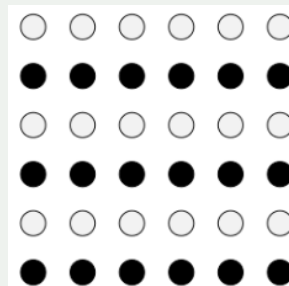
Wykorzystanie tzw. *praw (zasad) Gestalt* sformułowanych przez szkołę psychologii Gestalt na początku 20 wieku.

Psychologia Gestalt twierdzi, że prawa te istnieją, gdyż mózg ludzki ma naturalną tendencję do postrzegania wzorców/obiektów, które można zdefiniować na podstawie pięciu (sześciu?, siedmiu?) właściwości:

- Bliskość



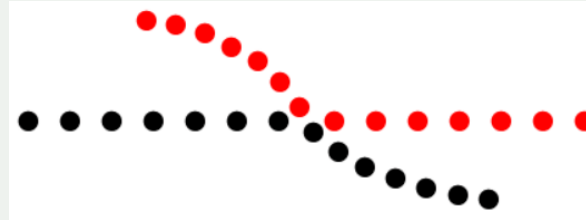
- Podobieństwo



Zasady projektowania interfejsów graficznych

Podstawy percepcji wizualnej

- Ciągłość (kontynuacja)



- Domknięcie (spójność percepcyjna)



- Symetria



Zasady projektowania interfejsów graficznych

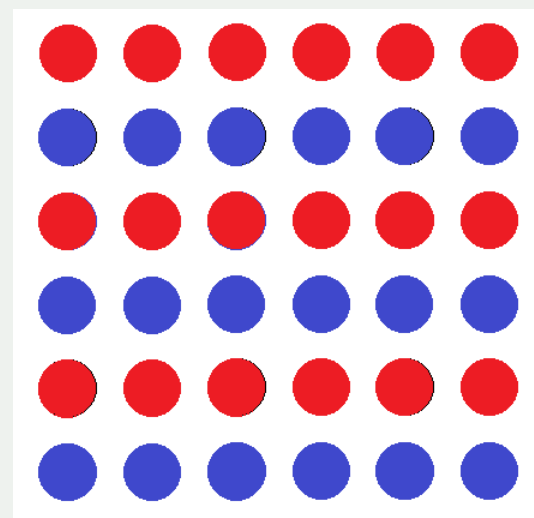
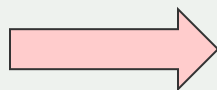
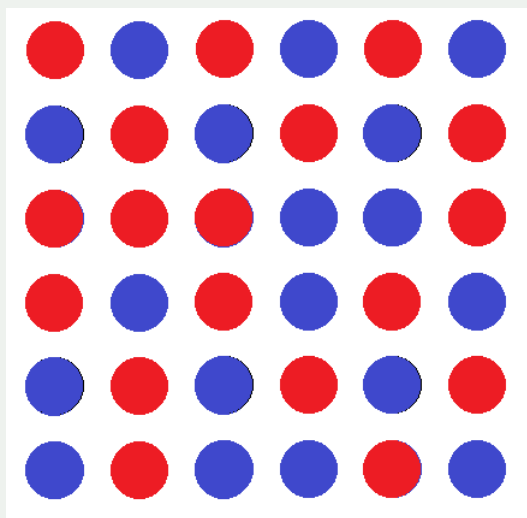
Podstawy percepcji wizualnej (przykład)

Używając tylko dwóch poniższych obiektów graficznych,

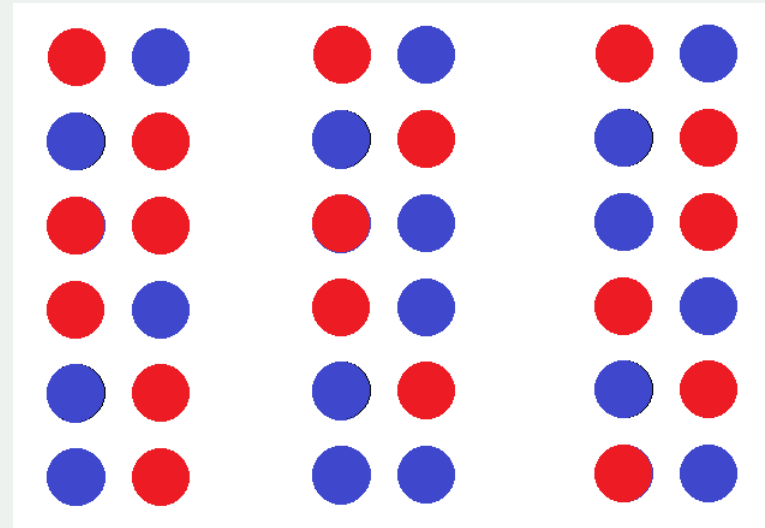
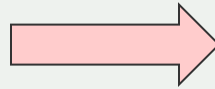
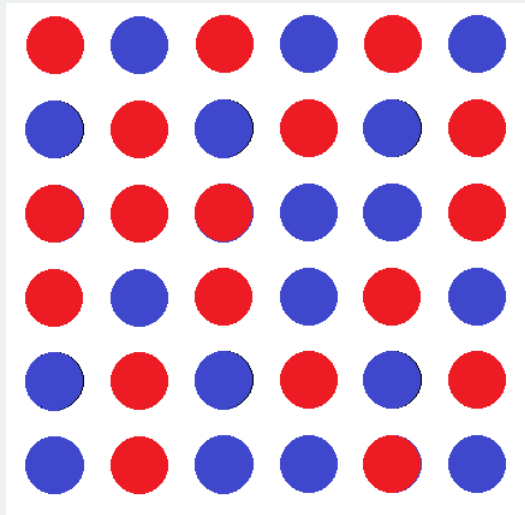


zbuduj z nich przykładowe rysunki (co najmniej po jednym do każdej zasady) ilustrujące jak każda z zasad Gestalt „pomaga” nam dostrzec w tych rysunkach konkretne fragmenty.

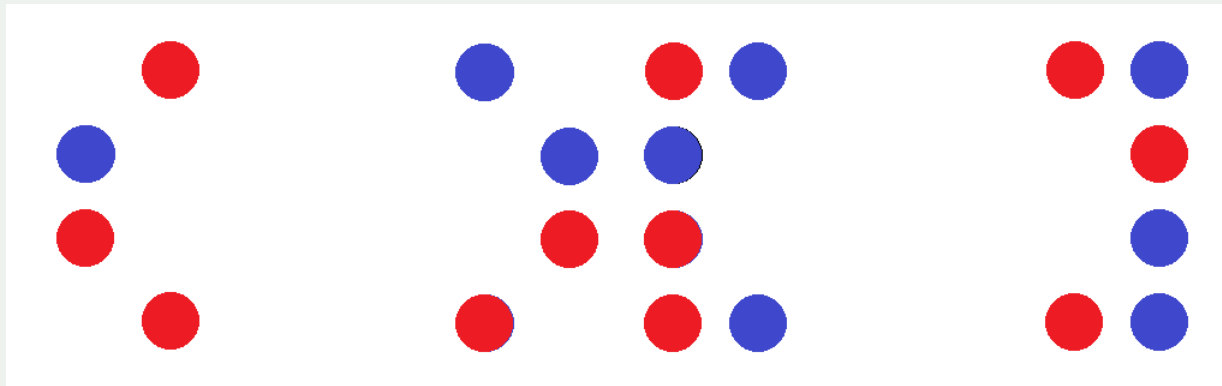
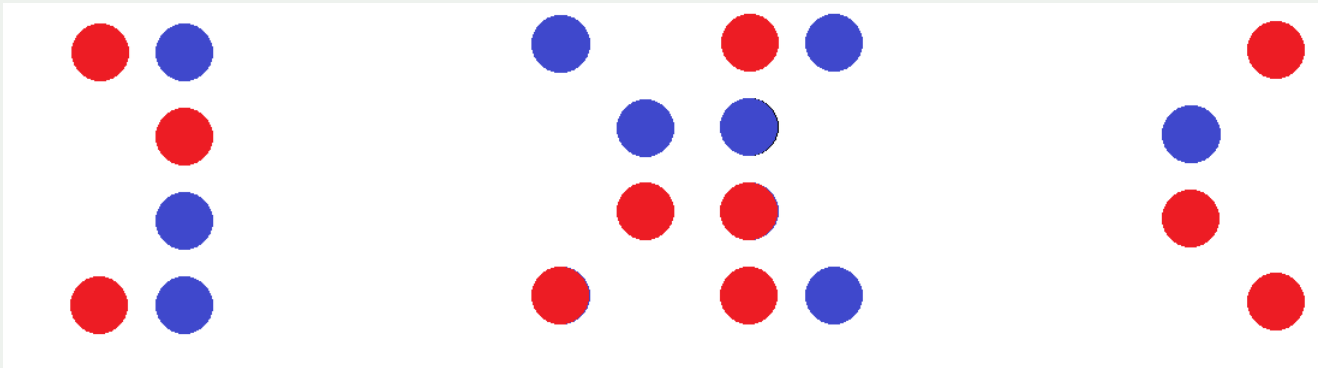
- Podobieństwo



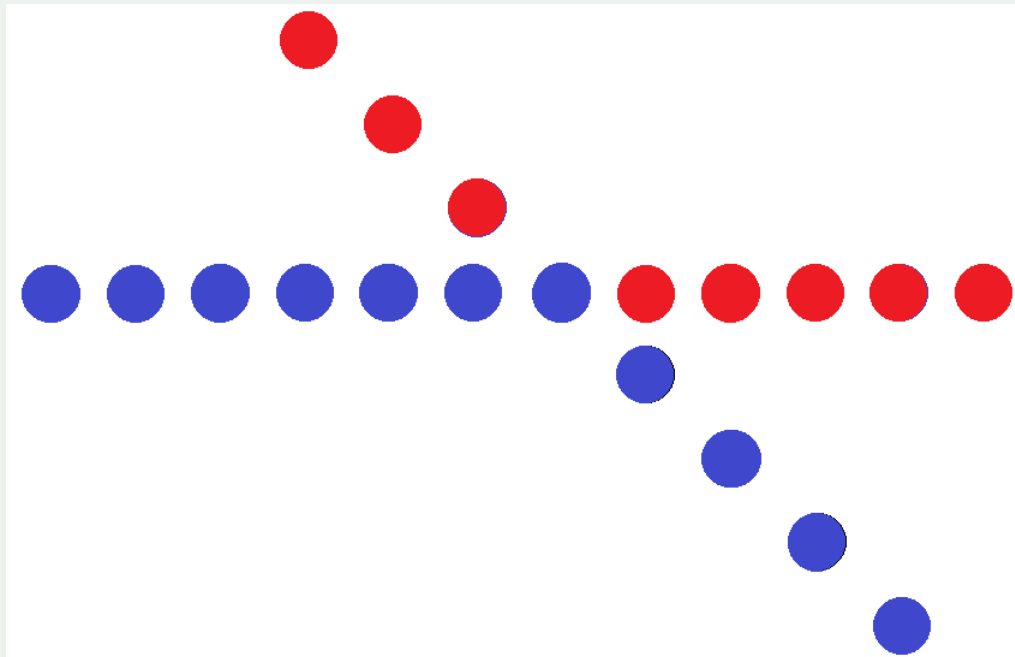
- Bliskość



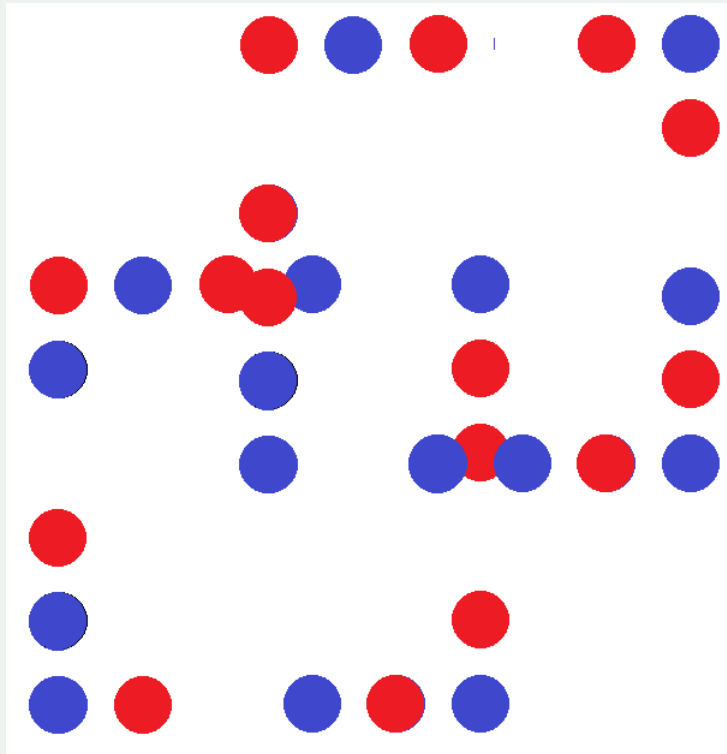
- Symetria



- Ciągłość (kontynuacja)



- Domknięcie (spójność percepcyjna)



Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

Funkcjonalność i ergonomia są często oceniane wg tzw. heurystyk Nielsena, 1990-94)

1. status systemu zawsze widoczny (możliwie najdogodniej dla użytkownika)
2. swoboda działania i sprawianie wrażenie, że system jest pod kontrolą (np. przewidywalny czas reakcji, dostępność do całej „przestrzeni roboczej”)
3. spójność (te same metody i standardy we wszystkich fragmentach/etapach)
4. ułatwione rozpoznawanie (np. ikon) zamiast ciągłego przypominania ich znaczenia
5. przewidywanie sytuacji prowadzących do powstawania błędów i zapobieganie takim sytuacjom
6. właściwa terminologia (nie wykraczająca poza wiedzę użytkownika)
7. elastyczne metody interakcji dopasowane do wymagań użytkownika (np. skróty)
8. estetyka i „oszczędność” graficzna
9. diagnostyka i korekcja błędów (czytelny opis błędów, operacja *undo*)
10. dokumentacja i pomoc (łatwo dostępna pomoc na żądanie)

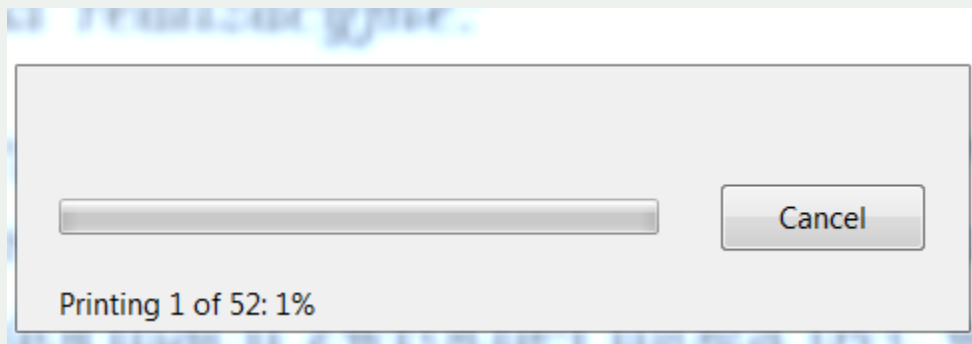
Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

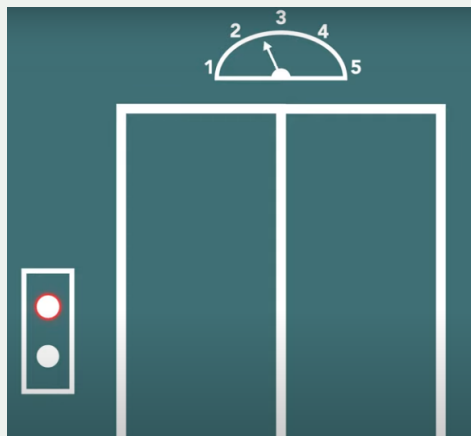
1. Status systemu zawsze widoczny (możliwie najdogodniej dla użytkownika)

Na przykład:

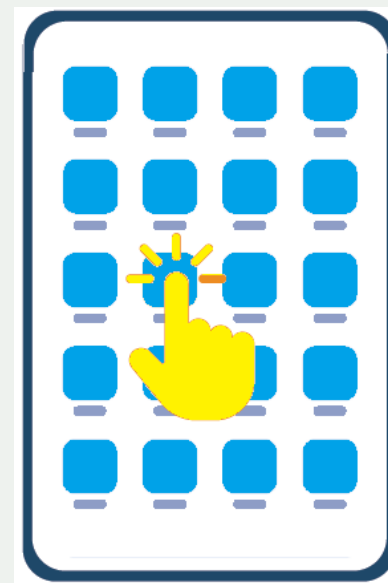
- Informacja zwrotna dostępna w najszerszym zakresie:



Pasek



Klepsydra



Dźwięk lub
podświetlenie

Zasady projektowania interfejsów graficznych

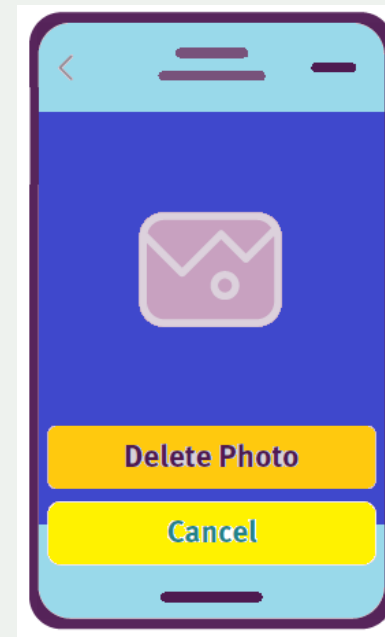
Wykorzystanie heurystyk Nielsena

2. Swoboda działania i sprawianie wrażenie, że system jest pod kontrolą (dostępność całej „przestrzeni roboczej”, wyjścia awaryjne, itp.)

Na przykład:



Operacje *undo* i *redo* (tutaj dość źle zrealizowane)



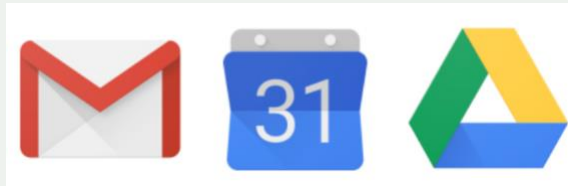
Dostępność przestrzeni roboczej

Zasady projektowania interfejsów graficznych

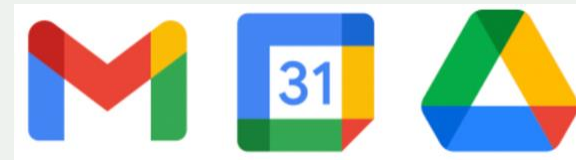
Wykorzystanie heurystyk Nielsena

3. Spójność (te same metody i standardy we wszystkich fragmentach/etapach)

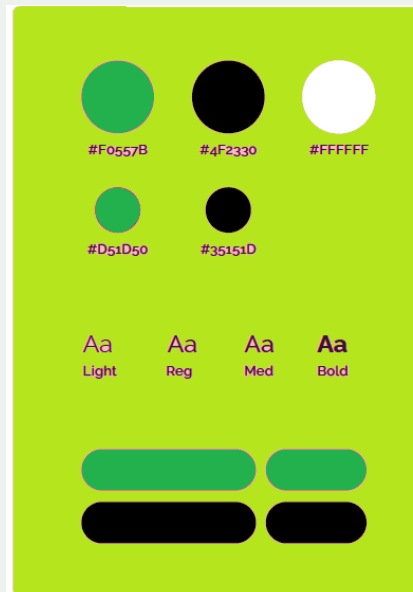
Na przykład:



Tak sobie



Dobrze



Kolor
Symbol
Czcionka

Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

4. Ułatwione rozpoznawanie zamiast ciągłego przypominania znaczenia

np. właściwe metafory (ikony)

Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

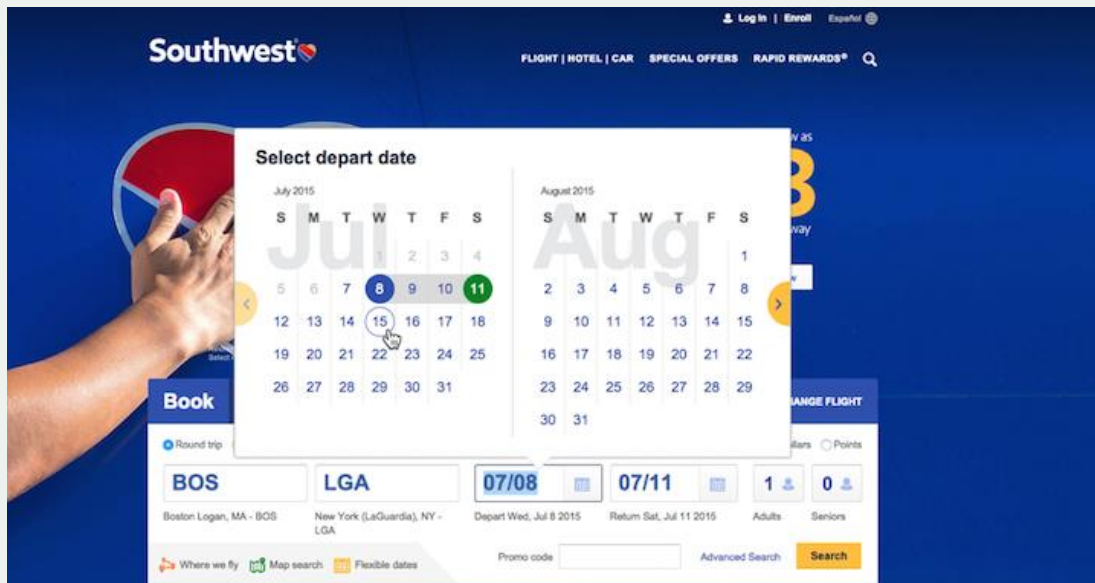
5. Przewidywanie sytuacji prowadzących do powstawania błędów i zapobieganie takim sytuacjom

Są dwa rodzaje błędów:

1. Pomyłki, ang. *slips* (popołnione nieświadomie).
2. Błędy, ang. *mistakes* (popołnione świadomie, np. niezrozumienie modelu)

Najbardziej „kosztowne” błędy/pomyłki powinny mieć najwyższy priorytet.

Pomyłek unikamy głównie poprzez przypominanie (tym częściej i natrętniej, im potencjalnie „kosztowniejsze” skutki).



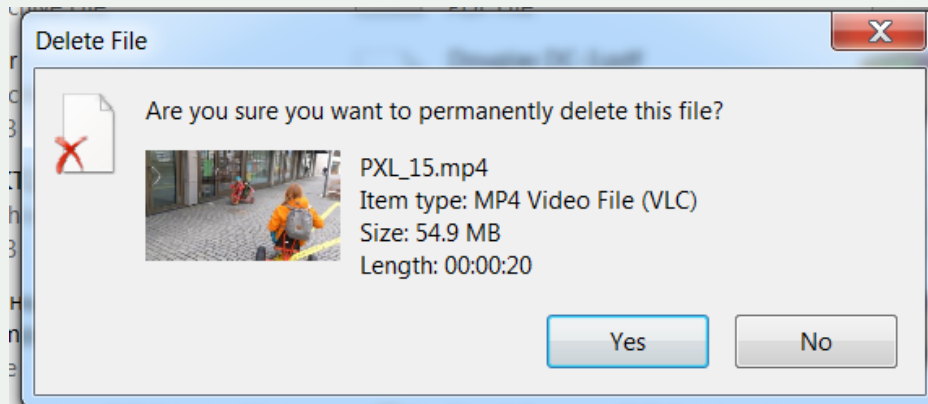
<https://www.nngroup.com/articles/slips/>

Zasady projektowania interfejsów graficznych

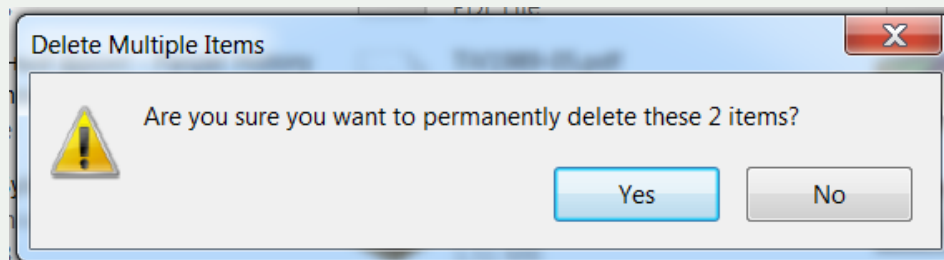
Wykorzystanie heurystyk Nielsena

5. Przewidywanie sytuacji prowadzących do powstawania błędów i zapobieganie takim sytuacjom

Potencjalne błędy (zwłaszcza błędy krytyczne) przedstawiane w postaci wyraźnych ostrzeżeń.



Dobre rozwiązanie



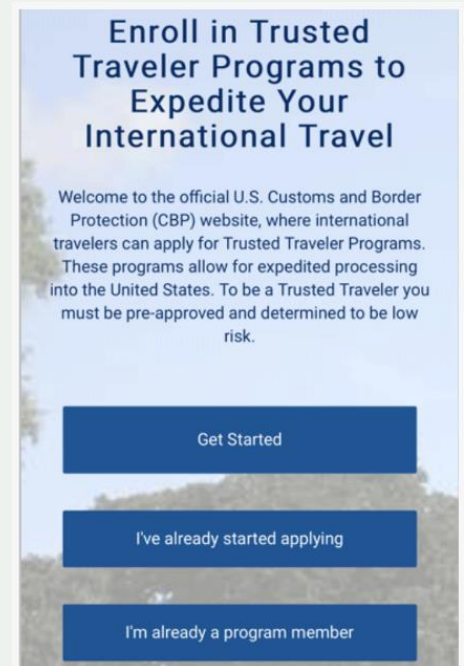
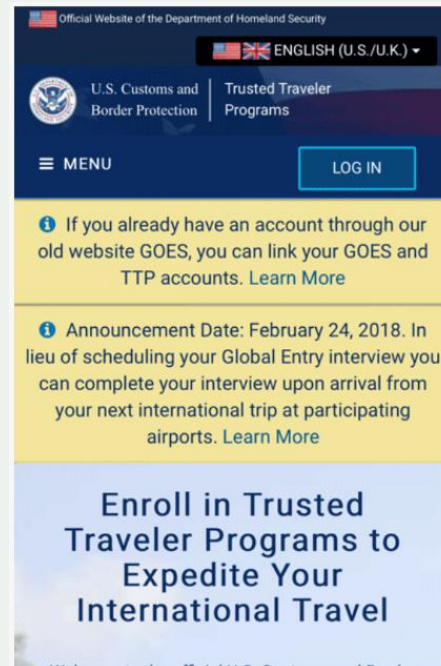
Gorsze rozwiązanie

Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

6. Właściwa terminologia (zgodna ze „światem realnym”, nie wykraczająca poza wiedzę użytkownika)

Na przykład:



<https://www.nngroup.com/articles/match-system-real-world/>

Złe rozwiązanie i propozycja lepszej grafiki.

Zasady projektowania interfejsów graficznych

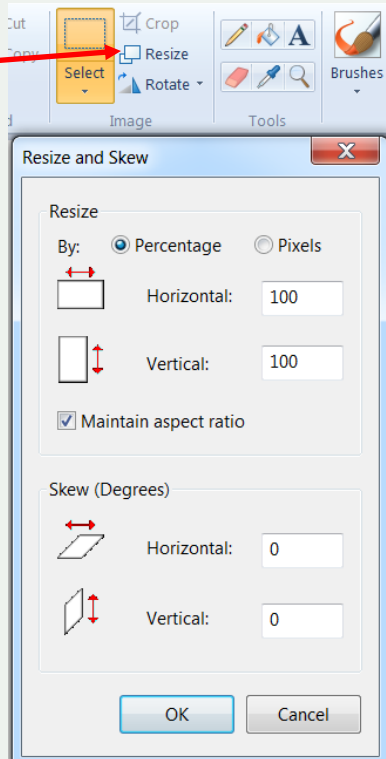
Wykorzystanie heurystyk Nielsena

7. Elastyczne metody interakcji dopasowane do wymagań użytkownika (np. skróty)

Oznacza zwykle *rezygnację* z pewnych elementów wizualizacji (zaawansowany użytkownik nie musi widzieć, żeby wiedzieć co zrobić).

Na przykład:

Zamiast wybierać
wciskamy CTRL+W



UWAGI:

„Kastomizacja” nie jest zalecanym rozwiązaniem, ponieważ większości użytkowników nie będzie się chciało tego robić,

Preferowana jest *personalizacja*, tzn. system automatycznie robi „kastomizację” na podstawie obserwacji zachowań użytkownika, ale....

personalizacja jest trudniejsze do realizacji i może być źródłem frustracji, jeśli zrobiona jest źle.

Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

8. Estetyka i „oszczędność” graficzna

Na przykład:

ESTETYKA

ESTETYKA

Forma graficzna powinna być skupiona wokół głównego przekazu.

Elementy graficzne powinny wspierać podstawowe zadanie danego interfejsu, a nie rozpraszać uwagę użytkownika.

Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

9. Diagnostyka i korekcja błędów (czytelny opis błędu, operacja *undo*, itd.)

Diagnostyka powinna wykryć każdy błąd. Najgorszy komunikat o błędzie jest lepszy, niż brak komunikatu, gdy błąd rzeczywiście wystąpił.

Czytelny język. Unikaj komunikatów typu „wystąpił błąd nr 15”.

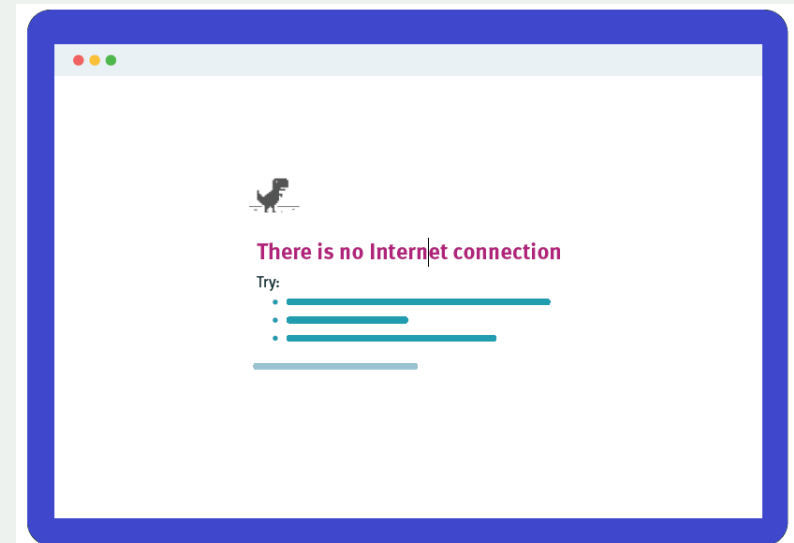
Uprzejmy ton. Nie sugeruj, że użytkownik jest głupi lub zrobił coś niewłaściwego, np. „nielegalna instrukcja”.

Dokładny opis błędu, zamiast ogólników w rodzaju „błąd syntaktyczny”.

Rada jak naprawić/poprawić sytuację.

UWAGA: W diagnostyce i korekcji błędu rzadko stosuje się elementy graficzne.

Wyjątkiem są systemy przedstawiane w postaci diagramów blokowych, na których można wyświetlać lokalizację błędu.



Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

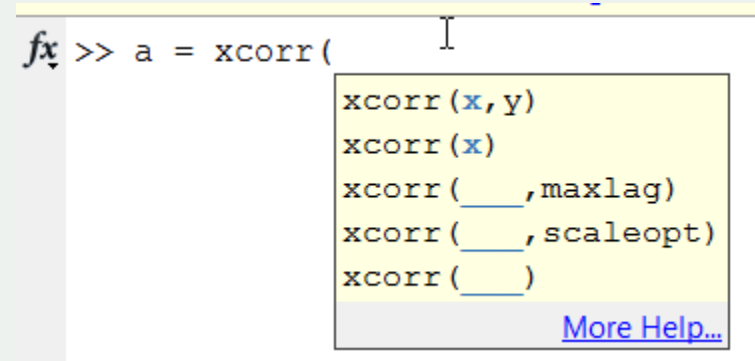
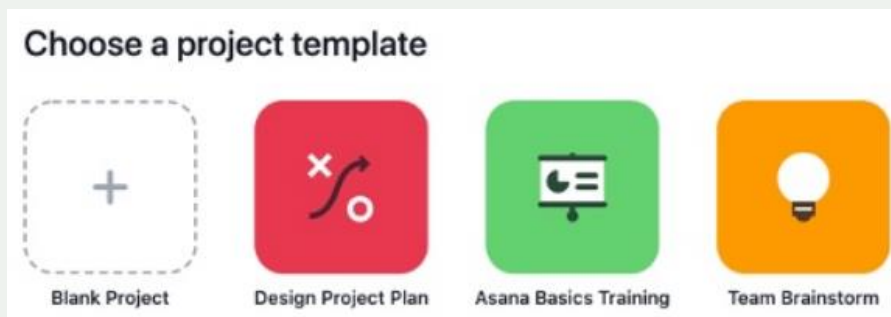
10. Dokumentacja i pomoc (łatwo dostępna pomoc na żądanie)

Pomoc *proaktywna* i *reaktywna*.

Celem pomocy proaktywnej jest:

1. Szkolenie nowych użytkowników, np. tutoriale, wzorce, *wizards* (asystenci, kreatorzy), itp.
2. Pomoc początkującym użytkownikom w osiągnięciu biegłości.
3. Pomoc w nowych/zmienionych możliwościach lub opcjach systemu.

Przykłady pomocy proaktywnej:



Zasady projektowania interfejsów graficznych

Wykorzystanie heurystyk Nielsena

10. Dokumentacja i pomoc (łatwo dostępna pomoc na żądanie)

Pomoc *reaktywna* powinna być dostępna jedynie na wyraźne żądanie użytkownika (gdy napotka on problem). Powinna wyjaśnić problem, odpowiedzieć na pytania i ogólnie pomóc użytkownikowi w osiągnięciu poziomu eksperta.

Najbardziej typowe formy pomocy reaktywnej to:

- FAQs.
- Dokumentacja techniczna. Pamiętaj, że użytkownicy nie lubią czytać dokumentacji, a jeśli już muszą, to oczekują tam specjalistycznej wiedzy (a nie ogólnej).
- Moduły/sesje treningowe.

Choć pomoc reaktywna jest głównie w formie tekstowej, można ją wspomagać grafiką i filmami.

UWAGA: Idealny system to taki, który nie wymaga dodatkowych wyjaśnień, a więc rozmiar dokumentacji jest nieformalną miarą niedoskonałości systemu (z punktu widzenia użytkownika).

Zasady projektowania interfejsów graficznych

Iluzje optyczne (potencjalne utrudnienia albo ułatwienia ☺ w odbiorze informacji wizualnej)

Najważniejsze rodzaje iluzji:

- Postrzeganie rozmiarów.
- Postrzeganie jasności (kolorów).
- Postrzeganie głębi.

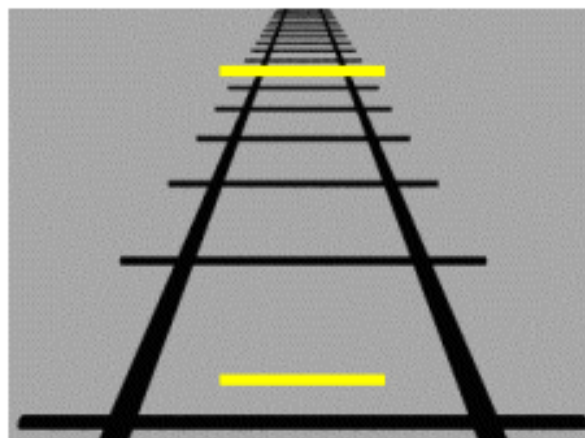
Inne rodzaje iluzji

- Postrzeganie (rozpoznawanie) kształtów.
- Postrzeganie struktur 3-wymiarowych (obiekty niemożliwe).
- Postrzeganie ruchu.

- Postrzeganie rozmiarów.



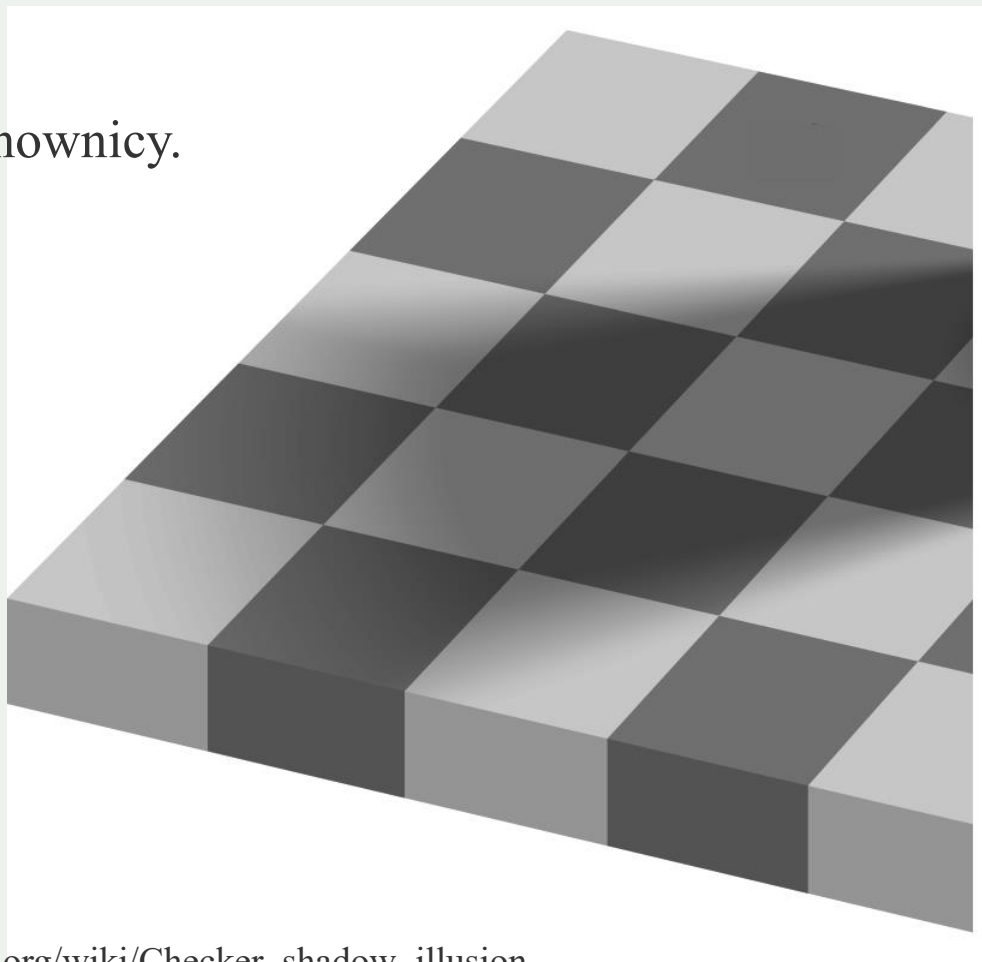
- Postrzeganie rozmiarów.



Iluzja Ponzo.

- Postrzeganie jasności (dotyczy również kolorów).

Iluzja szachownicy.

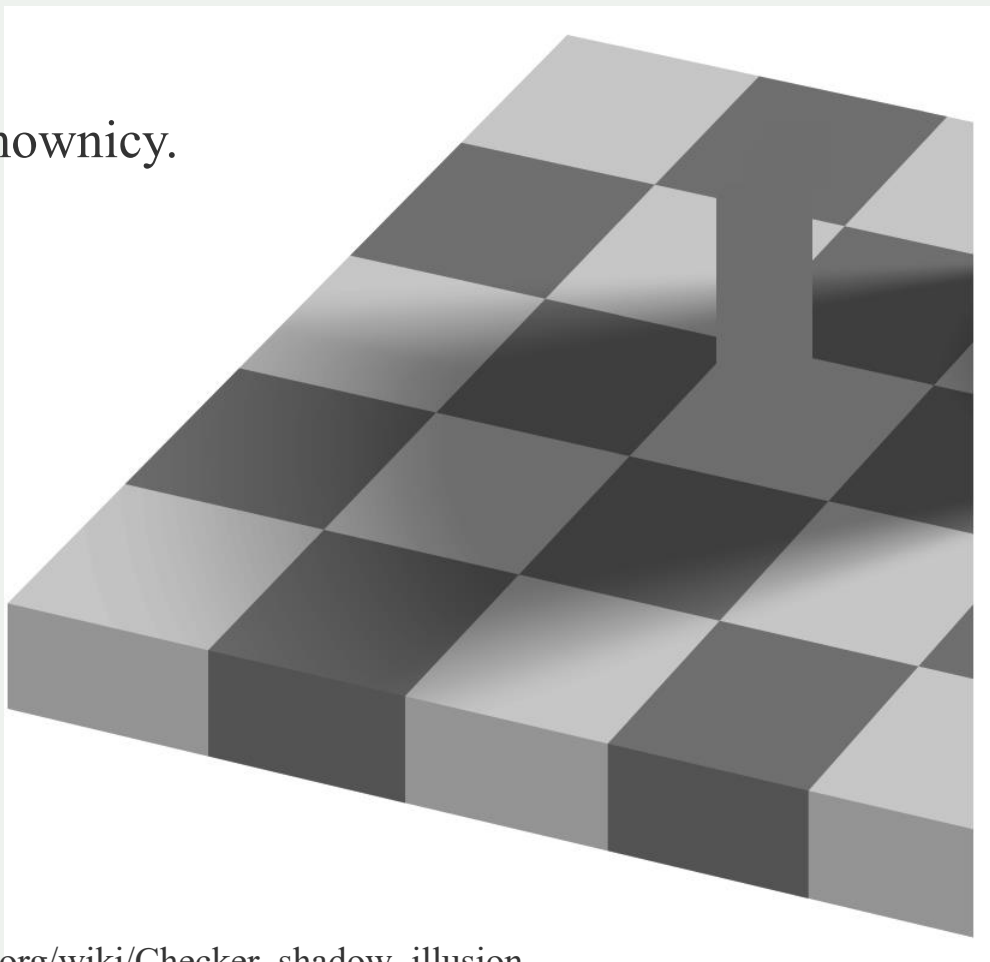


Edward H. Adelson

https://en.wikipedia.org/wiki/Checker_shadow_illusion

- Postrzeganie jasności (dotyczy również kolorów).

Iluzja szachownicy.



Edward H. Adelson

https://en.wikipedia.org/wiki/Checker_shadow_illusion

- Postrzeganie głębi.



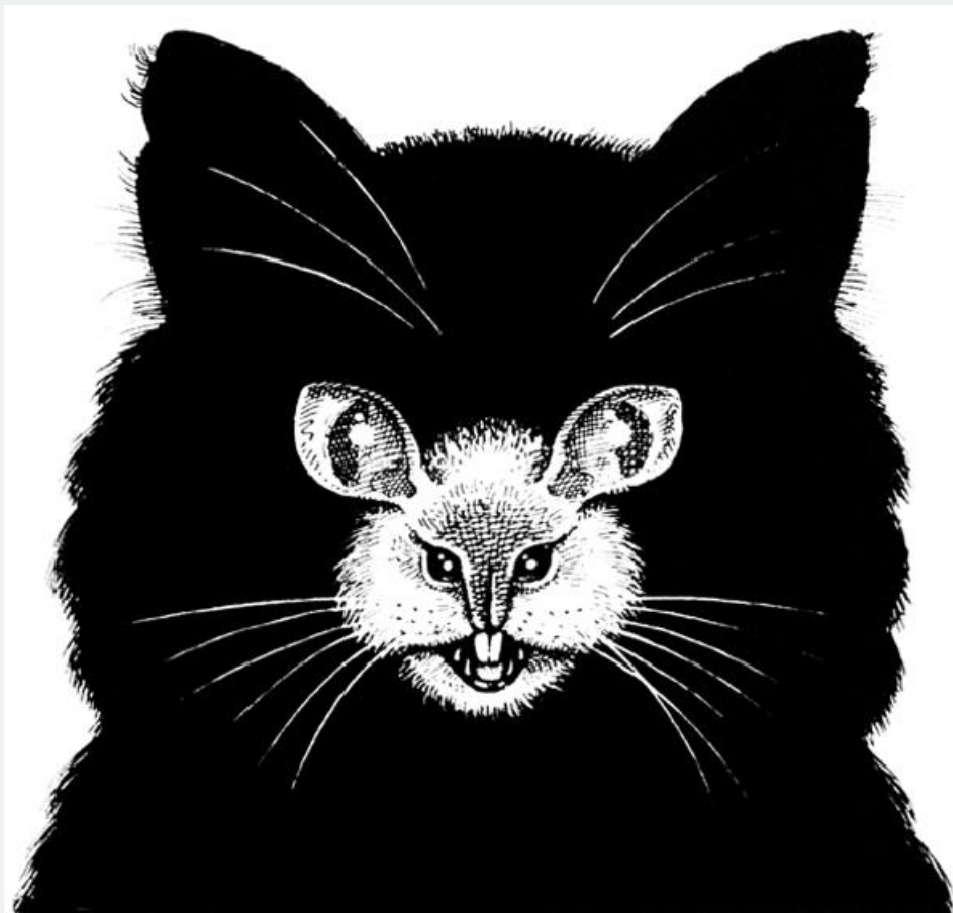
Iluzja „księżycowa”.

- Postrzeganie głębi.



Iluzja „księżycowa”.

- Postrzeganie (rozpoznawanie) kształtów.



Źródło: <https://designpress.com/inspiration/scary-optical-illusions/>

Zasady projektowania interfejsów graficznych

Podstawowe założenia

1. Wymagania wynikające z charakteru zadania

- Zastosowanie
- Funkcjonalność i ergonomia
- Zgodność ze standardami (formalnymi i nieformalnymi)

2. Profil użytkownika

- Ograniczenia (np. warunki eksploatacji, osoby niepełnosprawne, itd.)
- Możliwości percepcyjne i pamięciowe użytkownika
- Charakterystyka nawyków i typowych źródeł błędów

3. Realizacja projektu

- Sposoby pozyskiwania i przekazywania informacji
- Wykorzystanie istniejących rozwiązań
- Bieżąca współpraca z użytkownikami i możliwości testowania (UCD)

4. Możliwości rozbudowy (pod kątem przyszłych zastosowań)

- Dodawanie nowych funkcji i usług
- Otwartość na nowinki technologiczne (np. stacjonarne => mobilne)
- Otwartość na zmiany wymagań użytkownika